hashlock.

2026

SecurityAudit

Orbis (DeFi)

June 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

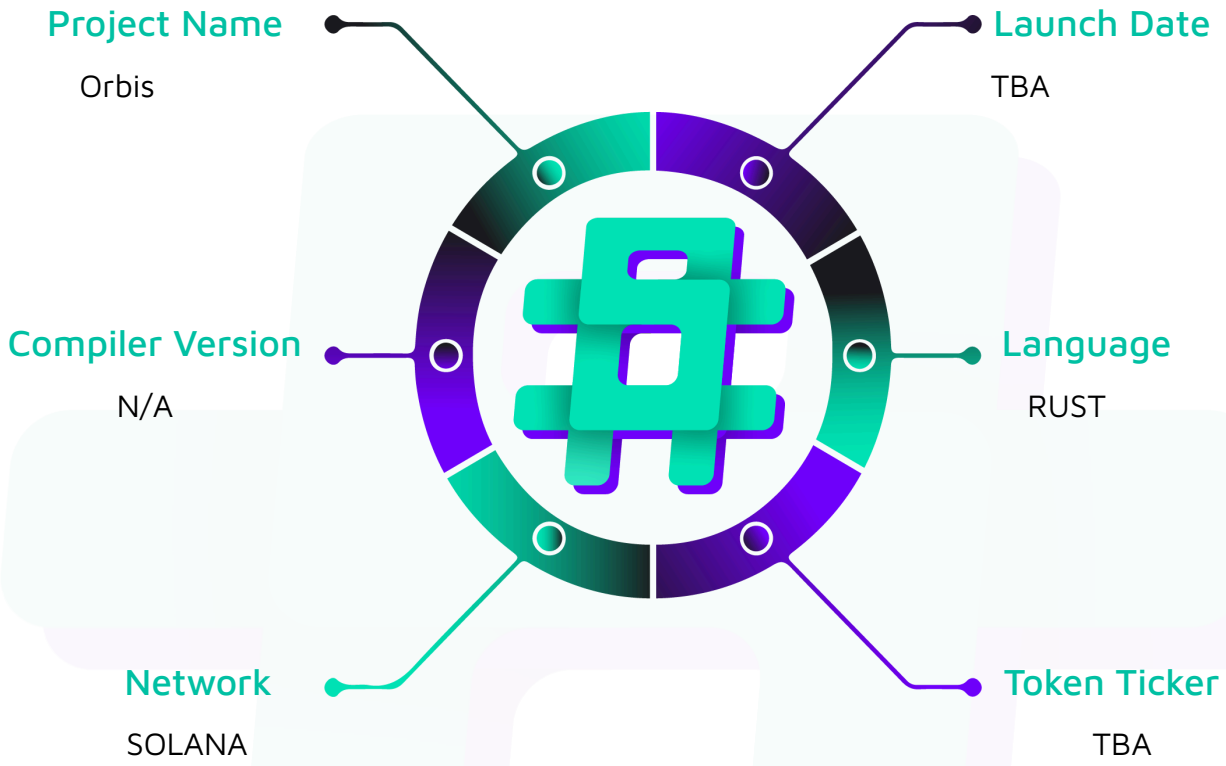
The Orbis team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

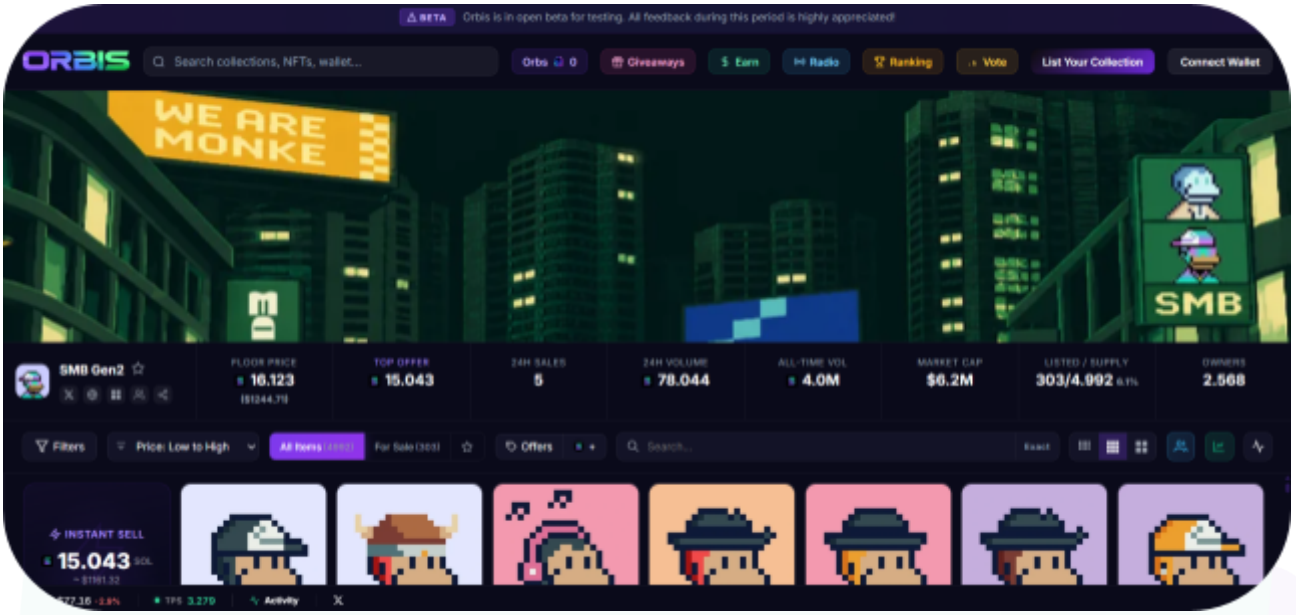
Orbis is a next-generation NFT marketplace built on Solana, focused on creating a more engaging, transparent, and value-driven trading experience for both collectors and project teams. Unlike traditional marketplaces, Orbis integrates native ecosystem mechanics such as marketplace fee redistribution, token buybacks, and community-driven incentives to align user activity with long-term platform growth.
Designed for speed and efficiency, Orbis leverages Solana's low-cost infrastructure to enable seamless minting, trading, and collection discovery while supporting emerging and established projects. The platform places strong emphasis on creator partnerships, offering tailored launch support, visibility tools, and ongoing ecosystem integration to help projects sustain traction beyond mint.
At its core, Orbis combines marketplace functionality with gamified engagement and reward systems, encouraging active participation from traders, holders, and communities. By tying platform usage to tangible ecosystem benefits, Orbis aims to redefine how value is created and distributed within the Solana NFT space

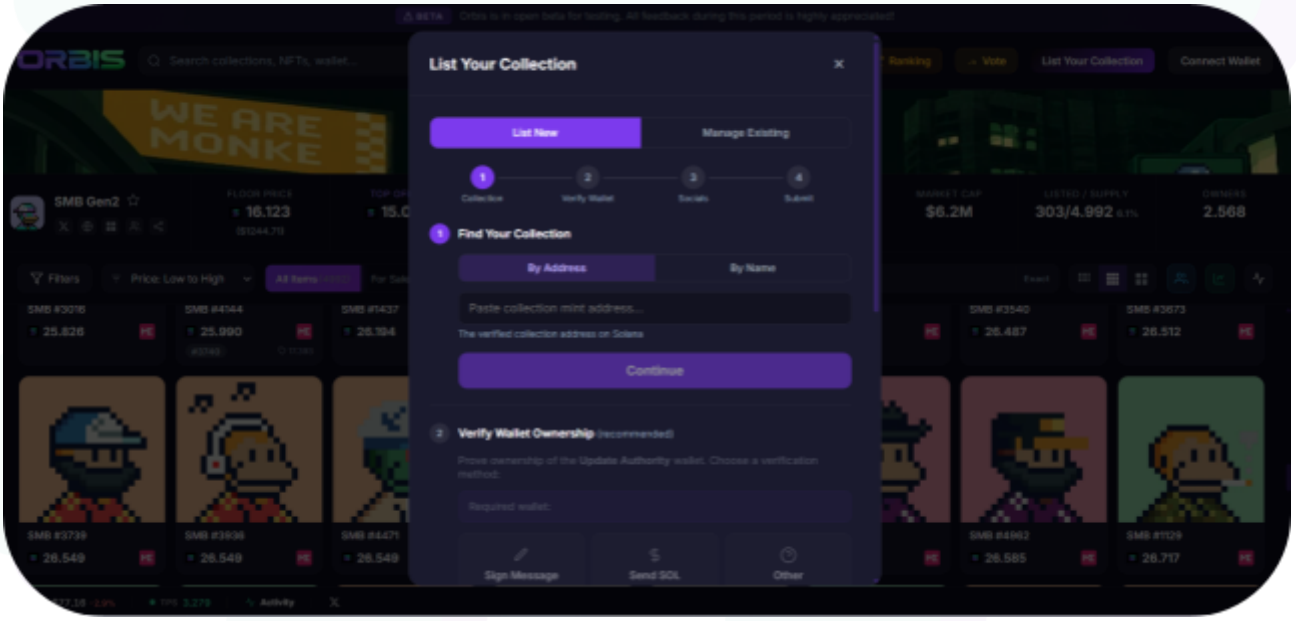
Project Name: Orbis
Project Type: Defi
Compiler Version: N/A
Website: https://www.orbisonsol.io/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Rust code within the Orbis project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the

smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Orbis Smart Contracts
Platform	Ethereum / Rust
Audit Date	June, 2026
Contract 1	utils.rs
Contract 2	lib.rs
Contract 3	instructions/accept_collection_offer_v2.rs
Contract 4	instructions/accept_offer.rs
Contract 5	instructions/buy_nft.rs
Contract 6	instructions/accept_collection_offer_v2_listed.rs
Contract 7	instructions/buy_core_v2.rs
Contract 8	instructions/list_nft.rs
Contract 9	instructions/list_core_v2.rs
Contract 10	instructions/list_cnft.rs
Contract 11	instructions/delist_cnft.rs
Contract 12	state/listing.rs
Contract 13	state/marketplace.rs
Contract 14	state/project.rs
Audited GitHub Commit Hash	2BdsaSDtY
Fix Review GitHub Commit Hash	E48a97d16

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

8 Medium severity vulnerabilities

12 Low severity vulnerabilities

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
instructions/accept_collection_offer_v2.rs: - Accept a collection offer for an unlisted NFT. - Transfer the NFT directly from seller to buyer. - Distribute escrowed SOL from the CollectionOffer PDA.	**Contract achieves this functionality.**
instructions/accept_collection_offer_v2_listed.rs: - Accept a collection offer for an already listed NFT. - Transfer the NFT from escrow or marketplace custody to the buyer. - Close both the listing and collection-offer accounts after settlement.	**Contract achieves this functionality.**
instructions/accept_offer.rs: - Accept an offer made on a listed NFT. - Transfer the NFT from escrow or marketplace custody to the buyer. - Distribute escrowed SOL to the seller, fee wallets, and royalty creators.	**Contract achieves this functionality.**
instructions/buy_core_v2.rs: - Buy a delegate-listed Core NFT. - Use marketplace delegate authority to thaw and transfer the Core asset. - Distribute sale proceeds, fees, and Core royalties.	**Contract achieves this functionality.**
instructions/buy_nft.rs: - Buy a listed legacy NFT, pNFT, Core NFT, or cNFT.	**Contract achieves this functionality.**

- Transfer the asset from escrow or marketplace custody to the buyer. - Distribute SOL to seller, platform, project, and royalty recipients.	
instructions/list_nft.rs: - Create a listing for a legacy NFT, pNFT, or escrow-mode Core NFT. - Move the asset into marketplace-controlled custody. - Record listing price, seller, project, type, and optional private buyer.	**Contract achieves this functionality.**
instructions/list_core_v2.rs: - Create a delegate-mode Core listing. - Read current Core plugin state on-chain. - Add or update marketplace FreezeDelegate and TransferDelegate authority.	**Contract achieves this functionality.**
instructions/list_cnft.rs: - Create a listing PDA for a compressed NFT. - Verify the companion Bubblegum transfer in the same transaction. - Consume the cosigner approval PDA and record Merkle tree data.	**Contract achieves this functionality.**
instructions/delist_nft.rs: - Delist an escrow-mode legacy NFT, pNFT, Core NFT, or cNFT. - Return the asset from escrow or marketplace custody to the seller. - Close the listing PDA and refund rent to the seller.	**Contract achieves this functionality.**

instructions/delist_core_v2.rs: - Delist a delegate-mode Core NFT. - Thaw the Core asset and remove or revoke marketplace plugin authority. - Close the listing PDA and decrement active listings.	**Contract achieves this functionality.**
instructions/delist_cnft.rs: - Delist a compressed NFT. - Transfer the cNFT leaf owner back from marketplace authority to seller. - Close the listing PDA and refund rent to the seller.	**Contract achieves this functionality.**
utils.rs: - Provide shared fee and royalty calculation helpers. - Validate metadata, creator shares, Core collection membership, and royalty overrides. - Define marketplace-wide fee and offer-expiry constants.	**Contract achieves this functionality.**
lib.rs: - Declare the program ID and public Anchor instruction entrypoints. - Define structured events used by off-chain indexers. - Route each public program method to its instruction handler.	**Contract achieves this functionality.**
state/listing.rs: - Store active listing data for a mint or cNFT asset ID. - Track seller, price, project, NFT type, offer	**Contract achieves this functionality.**

display state, private buyer, and cNFT Merkle metadata. - Define NFT type constants and listing mode constants.	
state/project.rs: - Store collection-level marketplace configuration. - Track project fees, offer settings, cosigner, counters, and optional Core collection support. - Define project PDA seeds and serialized size.	**Contract achieves this functionality.**
state/marketplace.rs: - Store global marketplace authority and platform fee configuration. - Define marketplace PDA seeds and marketplace-authority PDA seed. - Define the serialized size of the marketplace singleton.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Orbis project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Orbis project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] src/instructions/accept_collection_offer_v2_listed.rs#handler - Listed collection-offer acceptance ignores private listing targets and typed offer requirements

Description

The listed collection-offer acceptance path does not enforce the listing's private buyer restriction and does not check that a typed collection offer is fulfilled with the NFT type requested by the buyer.

Vulnerability Details

The `accept_collection_offer_v2_listed` handler loads both the `CollectionOffer` and `Listing` accounts, but it derives the transfer path from `listing.nft_type` and never enforces `listing.taker`.

```
pub fn handler<'info>(
    ctx: Context<'_, '_, '_, 'info, AcceptCollectionOfferV2Listed<'info>>,
    cnft_params: Option<CnftParams>,
    royalty_bps_override: Option<u16>,
) -> Result<()> {
    let offer = &ctx.accounts.collection_offer;
    let listing = &ctx.accounts.listing;
    let marketplace = &ctx.accounts.marketplace;
    let project = &ctx.accounts.project;
    let nft_type = listing.nft_type;
    // Verify offer hasn't expired
    let clock = Clock::get()?;
    require!(offer.expires_at > clock.unix_timestamp,
MarketplaceError::OfferExpired);
    // Verify buyer matches
    require!(ctx.accounts.buyer.key() == offer.buyer,
```

```
MarketplaceError::Unauthorized);
    // Verify listing mint matches the NFT
    require!(listing.mint == ctx.accounts.mint.key(),
MarketplaceError::MintMismatch);
    // For specific offers: verify mint matches
    if !offer.is_global {
        require!(ctx.accounts.mint.key() == offer.mint,
MarketplaceError::MintMismatch);
    }
```

The handler verifies that the transaction's buyer matches `offer.buyer` and that the listed mint is delivered, but it omits the private-listing rule that direct buys apply.

```
if let Some(taker) = listing.taker {
        require!(
            ctx.accounts.buyer.key() == taker,
            MarketplaceError::PrivateListing
        );
    }
```

The listed collection-offer path also does not compare `offer.nft_type` with `listing.nft_type`. The unlisted collection-offer path has an explicit typed-offer check:

```
if offer.nft_type != 255 {
        require!(
            nft_type == offer.nft_type,
            MarketplaceError::InvalidNftType
        );
    }
```

No equivalent check exists in `accept_collection_offer_v2_listed`. A seller can therefore accept a buyer's typed collection offer with a listed asset of a different standard. For example, if a buyer created a collection offer restricted to pNFTs, the listed accept path can fulfill it with a legacy NFT or Core asset from the same project as long as the listing account itself points to that asset.

Impact

Private listings do not remain private when they are fulfilled through a collection offer. A seller can bypass the intended `listing.taker` restriction and sell the asset to the collection-offer buyer instead of the private-listing target. Typed collection offers also lose their type guarantee, so buyers can receive an asset with transfer, royalty, or custody semantics different from the standard they explicitly requested.

Recommendation

At the beginning of `accept_collection_offer_v2_listed`, enforce the same private-listing check used by `buy_nft` and enforce the same typed-offer check used by `accept_collection_offer_v2`.

Add a `listing.taker` check mirroring the direct-buy private-listing rule, and add an `offer.nft_type != 255` check that compares `listing.nft_type` with `offer.nft_type` before transfer.

Status

Resolved

Medium

[M-01] src/instructions/transfer_marketplace_authority.rs#handler - Single-step marketplace authority transfer and live fee reads can silently change settlement terms

Description

The marketplace authority can be transferred to an arbitrary unchecked account in one step, and sale handlers use the current marketplace and project fee values at settlement time rather than values committed when a listing or offer was created.

Vulnerability Details

The `TransferMarketplaceAuthority` account context accepts `new_authority` as an `UncheckedAccount` and does not require that account to sign. The handler only rejects the default pubkey before writing the supplied key directly into `marketplace.authority`.

```
pub struct TransferMarketplaceAuthority<'info> {
    [..]
    pub authority: Signer<'info>,
    /// CHECK: The new authority wallet. Does not need to sign.
    pub new_authority: UncheckedAccount<'info>,
}
pub fn handler(ctx: Context<TransferMarketplaceAuthority>) -> Result<()> {
    let new_auth = ctx.accounts.new_authority.key();
    [..]
    require!(
        new_auth != Pubkey::default(),
        MarketplaceError::Unauthorized
    );
    let old_auth = ctx.accounts.marketplace.authority;
    let marketplace = &mut ctx.accounts.marketplace;
    marketplace.authority = new_auth;
    [..]
}
```

This makes the authority handoff a single irreversible write. A mistyped address, an address whose private key is not controlled, or an operational copy-paste error can permanently brick future marketplace administration because the destination is never required to prove key possession by accepting the transfer. This is an operational and theoretical liveness scenario rather than a clean external exploitation path, but the affected account is the marketplace singleton.

The `Listing`, `Offer`, and `CollectionOffer` accounts do not snapshot the fee terms that were visible when users entered the trade. The `Listing` account stores price and listing metadata, but it does not store `platform_fee_bps` or `project_fee_bps`.

```
pub struct Listing {
    /// Seller's wallet address
    pub seller: Pubkey,
    [..]
    /// Listing price in lamports
    pub price: u64,
    /// Reference to the Project PDA
    pub project: Pubkey,
    /// NFT type: 0=Legacy, 1=pNFT, 2=Core, 3=cNFT
    pub nft_type: u8,
    [..]
    /// Unix timestamp when listed
    pub created_at: i64,
    [..]
}
```

The `Offer` account stores the escrowed amount and timing metadata, but no fee snapshot.

```
pub struct Offer {
    /// Buyer's wallet address
    pub buyer: Pubkey,
    [..]
    /// Offer amount in lamports (escrowed in this PDA)
    pub amount: u64,
    /// Unix timestamp when offer expires
    pub expires_at: i64,
```

```
    /// Unix timestamp when offer was created
    pub created_at: i64,
    [..]
}
```

The `CollectionOffer` account follows the same pattern.

```
pub struct CollectionOffer {
    /// Buyer's wallet address
    pub buyer: Pubkey,
    /// Reference to the Project PDA (collection)
    pub project: Pubkey,
    [..]
    /// Offer amount in lamports (escrowed in this PDA)
    pub amount: u64,
    /// Unix timestamp when offer expires
    pub expires_at: i64,
    /// Unix timestamp when offer was created
    pub created_at: i64,
    [..]
}
```

Sale and accept paths calculate fees from the current marketplace and project state at the time of settlement.

```
let fees = calculate_fees(
        price,
        effective_royalty_bps,
        marketplace.platform_fee_bps,
        project.project_fee_bps,
    )?;
```

As a result, outstanding listings and escrowed offers are settled under whatever fee configuration is live when the sale executes, not under the fee configuration the seller or buyer saw when they created the listing or offer.

Impact

A single incorrect `new_authority` value can permanently lock marketplace administration until a program upgrade. Existing listings and offers can also settle under fee terms different from those that were visible at creation time: sellers receive less when fees increase and more when fees decrease. Both directions break fee transparency and make off-chain price displays unreliable for in-flight orders.

Recommendation

Implement a two-step authority-transfer flow. Store `pending_authority` in the `Marketplace` account and require a separate `accept_marketplace_authority_transfer` instruction signed by the pending authority before `marketplace.authority` changes.
Add fee snapshot fields to `Listing`, `Offer`, and `CollectionOffer`. At listing creation or offer creation, store the then-current `platform_fee_bps` and `project_fee_bps`. At settlement, use the snapshot instead of live marketplace and project state. Consider adding a delayed activation window for fee-wallet changes so users and indexers have time to react.

Status

Resolved

[M-02] src/instructions/list_core_v2.rs#handler - Unconstrained mpl_core_program enables spoofed Core CPIs during listing

Description

The Core listing paths accept `mpl_core_program` as an unchecked account without constraining it to the real MPL Core program ID. A caller can supply a different executable program for listing-time Core CPIs.

Vulnerability Details

The `list_core_v2` handler declares `mpl_core_program` as an `UncheckedAccount` and does not attach an `address = mpl_core::ID` account constraint.

```
/// CHECK: Metaplex Core program
    pub mpl_core_program: UncheckedAccount<'info>,
```

The handler then performs multiple Core CPIs through the caller-supplied account.

```
UpdatePluginV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info())
                .asset(&ctx.accounts.mint.to_account_info())
                .collection(Some(&ctx.accounts.core_collection.to_account_info()))
                .payer(&ctx.accounts.seller.to_account_info())

.authority(Some(&ctx.accounts.marketplace_authority.to_account_info()))
                .system_program(&ctx.accounts.system_program.to_account_info())
                .plugin(Plugin::FreezeDelegate(FreezeDelegate { frozen: true }))
                .invoke_signed(&[authority_seeds])?;
```

```
AddPluginV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info())
            .asset(&ctx.accounts.mint.to_account_info())
            .collection(Some(&ctx.accounts.core_collection.to_account_info()))
            .payer(&ctx.accounts.seller.to_account_info())
            .authority(Some(&ctx.accounts.seller.to_account_info()))
            .system_program(&ctx.accounts.system_program.to_account_info())
            .plugin(Plugin::FreezeDelegate(FreezeDelegate { frozen: true }))
            .init_authority(PluginAuthority::Address {
```

```
                address: marketplace_authority_key,
            })
```

The same issue exists in the Core branch of `list_nft`. The account is unchecked:

```
/// CHECK: Metaplex Core program (for Core NFTs)
    pub mpl_core_program: UncheckedAccount<'info>,
```

and the helper passes it directly into `CoreTransferV1CpiBuilder`.

```
fn escrow_core<'info>(ctx: &Context<'_, '_, '_, 'info, ListNft<'info>>) -> Result<()>
{
    CoreTransferV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info())
        .asset(&ctx.accounts.mint.to_account_info())
        .collection(Some(&ctx.accounts.core_collection.to_account_info()))
        .payer(&ctx.accounts.seller.to_account_info())
        .authority(Some(&ctx.accounts.seller.to_account_info()))
        .new_owner(&ctx.accounts.marketplace_authority.to_account_info())
        .system_program(Some(&ctx.accounts.system_program.to_account_info()))
        .invoke()?;
```

On Solana, a CPI builder invokes the program account supplied to it. If the caller can provide a program that accepts the instruction data and returns success without modifying the Core asset or plugin state, the marketplace can proceed as if the listing-time custody or freeze operation succeeded.

Impact

A seller can create a ghost Core listing: the marketplace initializes the `Listing` PDA and updates listing statistics, but the Core asset was not transferred, frozen, or delegated as expected. Such listings cannot be fulfilled correctly and can distort marketplace state, project active-listing counts, and off-chain floor or inventory views.

Recommendation

Add an account constraint to both listing contexts.

Use an account constraint such as `address = mpl_core::ID` on `mpl_core_program` in both

listing contexts.

Apply this to `list_core_v2.rs` and `list_nft.rs`. The `delist_nft.rs` file already has a runtime `mpl_core_program.key() == mpl_core::ID` check for the Core branch, so adding an account-level constraint there would be consistency hardening rather than core remediation.

Status

Resolved

[M-03] src/instructions/migrate_project.rs#handler - Project migration zero-fills new offer flags as disabled

Description

The `migrate_project` handler resizes v1 project accounts with zero initialization, causing new v2 collection-offer feature flags to deserialize as `false`. Migrated projects therefore lose collection-offer functionality until the authority manually re-enables the new flags.

Vulnerability Details

The current `Project` layout includes both legacy offer state and the newer granular collection-offer flags.

```
pub struct Project {
    /// Project owner wallet (can update settings)
    pub authority: Pubkey,
    /// NFT collection address
    pub collection: Pubkey,
    /// Project fee in basis points (e.g., 250 = 2.5%)
    pub project_fee_bps: u16,
    /// Wallet that receives project fees
    pub project_fee_wallet: Pubkey,
    /// Whether to enforce Metaplex creator royalties
    pub enforce_royalties: bool,
    /// Whether offers are enabled
    pub offers_enabled: bool,
    /// Minimum offer as basis points of listing price (e.g., 5000 = 50%)
    pub min_offer_bps: u16,
    /// Default offer expiry in seconds (e.g., 172800 = 48h)
    pub default_offer_expiry: i64,
    /// Number of currently active listings
    pub active_listings: u32,
    /// Total trade volume in lamports
    pub total_volume: u64,
    /// Total number of completed sales
    pub total_sales: u32,
    /// Whether the project is enabled
    pub enabled: bool,
```

```
    /// PDA bump
    pub bump: u8,
    /// Optional cosigner pubkey. When set, list/buy/accept_offer require this
cosigner.
    pub cosigner: Option<Pubkey>,
    /// Whether NFT-specific collection offers are enabled (offer on a specific mint
without listing)
    pub specific_offers_enabled: bool,
    /// Whether global collection offers are enabled (offer on any NFT in the
collection)
    pub global_offers_enabled: bool,
    /// Secondary Core collection for mixed-standard hashlist projects (e.g.
    /// Frens Factory: pNFT + MPL Core under one project). When set, Core
    /// list/buy/offer-accept paths accept an asset whose on-chain collection
    /// is EITHER `collection` OR `extra_core_collection`. Off-chain hashlist
    /// gating already limits which mints can reach a list path; this field
    /// loosens the on-chain collection check from a single equality to a
    /// two-key membership. `None` for single-standard projects (the default
    /// and the pre-existing behaviour for every project created before this
    /// field existed — uninitialised _reserved bytes deserialize as None).
    pub extra_core_collection: Option<Pubkey>,
    /// Reserved for future fields (avoids realloc)
    pub _reserved: [u8; 29],
}
```

Freshly created v2 projects explicitly enable the granular offer surfaces by default.

```
project.specific_offers_enabled = true;
    project.global_offers_enabled = true;
    // Secondary Core collection is unset at create time. Admins opt in via
    // update_project for mixed-standard hashlist projects only. See
    // Project.extra_core_collection doc-comment.
    project.extra_core_collection = None;
    project._reserved = [0u8; 29];
```

By contrast, migration only reallocates the old account and asks Solana to zero-initialize the appended region.

```
let current_len = project_info.data_len();
    let target_len = Project::SPACE;
    // Already at v2 size — no-op
    if current_len >= target_len {
        msg!("Project already at v2 size ({}), skipping", current_len);
        return Ok(());
    }
    {
        let data = project_info.try_borrow_data()?;
        // Verify Project account discriminator. Using the Anchor-derived
        // constant so the check follows the struct definition automatically
        // rather than relying on a hand-transcribed byte array.
        require!(
            data.len() >= 40,
            MarketplaceError::Unauthorized
        );
        require!(
            data[..8] == Project::DISCRIMINATOR,
            MarketplaceError::Unauthorized
        );
        // Verify caller is the project authority. The v1 Project layout
        // (136 bytes, pre-migration) stores `authority: Pubkey` as the first
        // field after the 8-byte discriminator. Anchor cannot deserialize the
        // old layout via the current `Project` struct, so this offset read
        // is load-bearing on v1 layout stability. If v1 is ever re-versioned
        // before full migration, revisit.
        let stored_authority = Pubkey::try_from(&data[8..40])
            .map_err(|_| error!(MarketplaceError::Unauthorized))?;
        require!(
            stored_authority == ctx.accounts.authority.key(),
            MarketplaceError::Unauthorized
        );
    }
    // Fund additional rent for the larger account
    let rent = Rent::get()?;
    let required_lamports = rent.minimum_balance(target_len);
    let current_lamports = project_info.lamports();
    if current_lamports < required_lamports {
```

```
        system_program::transfer(
            CpiContext::new(
                ctx.accounts.system_program.to_account_info(),
                system_program::Transfer {
                    from: ctx.accounts.authority.to_account_info(),
                    to: project_info.clone(),
                },
            ),
            required_lamports - current_lamports,
        )?;
    }
    // Resize account data and zero-initialise the new region atomically.
    // `realloc(_, true)` guarantees the appended bytes are 0x00, so the new
    // fields (cosigner = None, extra_core_collection = None, _reserved =
    // [0; 29]) deserialise correctly. The previous manual zero-fill pattern
    // relied on a follow-up borrow that, while always reached in practice
    // due to Solana's atomic transactions, was a fragile two-step where a
    // future refactor could leave the new bytes uninitialised between
    // realloc and the fill.
    project_info.realloc(target_len, true)?;
```

The new `specific_offers_enabled` and `global_offers_enabled` bytes are in the appended region. Since `false` deserializes as zero, migrated accounts differ from new accounts even when the old project had offers enabled.

The `make_collection_offer_v2` handler requires the granular flags and does not fall back to the old `offers_enabled` field.

```
if is_global {
        require!(project.global_offers_enabled,
MarketplaceError::GlobalOffersDisabled);
    } else {
        require!(project.specific_offers_enabled,
MarketplaceError::SpecificOffersDisabled);
    }
```

Impact

Every v1 project migrated with the current function starts with both collection-offer paths disabled. Buyers attempting to create global or specific collection offers receive `GlobalOffersDisabled` or `SpecificOffersDisabled` even if the project was offer-enabled before migration. This can silently break a launch or upgrade process until each project authority discovers the issue and submits a corrective `update_project` transaction.

Recommendation

After reallocating a v1 project, explicitly initialize all newly introduced semantic fields instead of relying on zero-fill. For migrated accounts, set `specific_offers_enabled` and `global_offers_enabled` to the intended migration default, likely matching the fresh-project default or deriving from the legacy `offers_enabled` flag. Emit a `ProjectMigrated` event that includes the initialized flag values.

Status

Resolved

[M-04] src/instructions/migrate_project.rs#handler - Lost v1 project authorities cannot be migrated despite documented admin recovery

Description

The migration handler requires the original project authority to sign and has no marketplace-authority fallback. Any v1 project whose authority key is lost cannot be migrated, even though the handover documentation says migration is marketplace-authority controlled.

Vulnerability Details

The `migrate_project` handler accepts an unchecked project account because the current Anchor `Project` type cannot deserialize the old v1 layout. It then manually reads the first post-discriminator field as the stored project authority.

```
#[derive(Accounts)]
pub struct MigrateProject<'info> {
    /// CHECK: Existing v1 project account. Owner, discriminator, and authority
validated in handler.
    #[account(mut)]
    pub project: UncheckedAccount<'info>,
    #[account(mut)]
    pub authority: Signer<'info>,
    pub system_program: Program<'info, System>,
}
```

```
let data = project_info.try_borrow_data()?;
        // Verify Project account discriminator. Using the Anchor-derived
        // constant so the check follows the struct definition automatically
        // rather than relying on a hand-transcribed byte array.
        require!(
            data.len() >= 40,
            MarketplaceError::Unauthorized
        );
        require!(
            data[..8] == Project::DISCRIMINATOR,
```

```
            MarketplaceError::Unauthorized
        );
        // Verify caller is the project authority. The v1 Project layout
        // (136 bytes, pre-migration) stores `authority: Pubkey` as the first
        // field after the 8-byte discriminator. Anchor cannot deserialize the
        // old layout via the current `Project` struct, so this offset read
        // is load-bearing on v1 layout stability. If v1 is ever re-versioned
        // before full migration, revisit.
        let stored_authority = Pubkey::try_from(&data[8..40])
            .map_err(|_| error!(MarketplaceError::Unauthorized))?;
        require!(
            stored_authority == ctx.accounts.authority.key(),
            MarketplaceError::Unauthorized
        );
```

Only the stored v1 authority can pass this check. There is no branch that accepts `marketplace.authority` or any other admin signer. That contradicts the handover statement that migration is only callable by the marketplace authority, and it removes the expected operational recovery path for abandoned or lost-key projects.

Because the upgraded program expects the v2 `Project::SPACE` layout, a v1 account that has not been migrated is only usable by the special migration path. Normal sale, delist, buy, and offer handlers expect the current `Project` account layout and will fail account deserialization or constraints for an old account.

Impact

If a v1 project's authority key is lost, that project is permanently stuck at the old account size. Existing escrowed assets associated with that project can become unusable through the upgraded program because normal handlers cannot operate on the old layout and migration cannot be authorized. Operators relying on the handover may also incorrectly assume the marketplace authority can recover these projects.

Recommendation

Reconcile the documentation and code. If admin recovery is intended, include the marketplace singleton in the migration context and allow migration when either the stored project authority signs or `marketplace.authority` signs. Since migration only

reallocates and initializes new fields, the admin fallback can be implemented without modifying the old economic fields. Emit a distinct event when the marketplace authority performs the migration.

Status

Resolved

[M-05] src/state/project.rs#enforce_royalties - Royalty enforcement toggle is written but never read by sale paths

Description

The `Project.enforce_royalties` field is configurable through project creation and update instructions, but sale handlers ignore it and always enforce on-chain royalties for non-cNFT standards.

Vulnerability Details

The project state defines `enforce_royalties` as a semantic toggle.

```
/// Whether to enforce Metaplex creator royalties
    pub enforce_royalties: bool,
```

Both project setters write the field.

```
project.enforce_royalties = enforce_royalties;
```

```
if let Some(enforce) = enforce_royalties {
        project.enforce_royalties = enforce;
    }
```

However, sale handlers do not consult this field when computing creator royalties. For legacy and pNFT assets, they read Token Metadata royalties unconditionally.

```
// Always enforce royalties — read from on-chain metadata regardless of project flag
        let rbps: u16 = metadata.seller_fee_basis_points;
```

```
let fees = calculate_fees(
        price,
        effective_royalty_bps,
        marketplace.platform_fee_bps,
        project.project_fee_bps,
    )?;
```

The field therefore creates a false configuration surface. A project authority can set `enforce_royalties = false`, but the resulting sale behavior is identical to `true`.

Impact

Project operators may believe they can opt out of creator royalty enforcement for a collection or promotional market, but the program continues to charge royalties on eligible standards. This creates integration risk, user-facing misconfiguration, and potentially failed sales when royalties plus fees exceed the program's total-fee ceiling even though the project thought royalties were disabled.

Recommendation

Either implement the field or remove it. If implemented, sale paths should set `effective_royalty_bps = 0` and skip creator payouts when `project.enforce_royalties == false`, subject to the desired product policy. If the marketplace always enforces royalties, remove the field from the account model and instruction arguments, and document the invariant clearly in the IDL and frontend.

Status

Resolved

[M-06] src/instructions/list_nft.rs#handler - High-royalty assets can be listed before sale-time fee ceiling rejection

Description

The program enforces the total-fee ceiling only during sale execution. It does not validate royalties plus platform and project fees at list time, so assets can be escrowed or frozen even though every later sale will revert unless fees or royalties change.

Vulnerability Details

The fee helper rejects sales when the sum of royalty, platform fee, and project fee exceeds `MAX_TOTAL_FEE_BPS`.

```
pub fn calculate_fees(
    sale_price: u64,
    royalty_bps: u16,
    platform_fee_bps: u16,
    project_fee_bps: u16,
) -> Result<FeeBreakdown> {
    // Enforce the combined-fee ceiling. Without this, a collection with
    // high on-chain royalties plus maxed platform+project fees could drive
    // the seller's proceeds to zero. Projects needing to trade
    // extreme-royalty collections must use the cosigner-gated royalty
    // override path to bring totals under MAX_TOTAL_FEE_BPS.
    let total_fee_bps = (royalty_bps as u32)
        .checked_add(platform_fee_bps as u32)
        .ok_or(MarketplaceError::ArithmeticOverflow)?
        .checked_add(project_fee_bps as u32)
        .ok_or(MarketplaceError::ArithmeticOverflow)?;
    require!(
        total_fee_bps <= MAX_TOTAL_FEE_BPS as u32,
        MarketplaceError::TotalFeesTooHigh
    );
```

Sale handlers call this after reading metadata or Core royalty data.

```
let fees = calculate_fees(
```

```
        price,
        effective_royalty_bps,
        marketplace.platform_fee_bps,
        project.project_fee_bps,
    )?;
```

Listing handlers, however, do not perform the same calculation before custody changes. Legacy and pNFT listings move the token into escrow. Core v2 listings add or update delegate plugins and freeze the asset. If royalty plus marketplace and project fees exceed the ceiling, the listing can exist but cannot be bought successfully.

```
let project = &mut ctx.accounts.project;
    project.active_listings = project
        .active_listings
        .checked_add(1)
        .ok_or(MarketplaceError::ArithmeticOverflow)?;
```

The same state can arise after listing if `platform_fee_bps` or `project_fee_bps` is increased and the new total exceeds the ceiling for outstanding listings.

Impact

Sellers can escrow or freeze assets into listings that are functionally unsaleable. Legacy and pNFT sellers must delist to recover escrowed assets. Core sellers must use the correct delegate-mode delist path. This does not affect cNFT royalties because the current cNFT sale paths hardcode royalty BPS to zero, but it affects legacy, pNFT, and Core assets whose on-chain royalty configuration can push total fees above the sale-time limit.

Recommendation

Read royalty data at list time and enforce the same ceiling before custody or plugin changes occur.

Apply a list-time equivalent of the existing `calculate_fees` total-fee ceiling before escrow or delegate-state changes occur.

Also apply compatibility checks in `update_marketplace` and `update_project` when raising

platform or project fees, or use listing-level fee and royalty snapshots so outstanding listings are not invalidated by later fee changes.

Status

Resolved

[M-07] src/instructions/make_collection_offer_v2.rs#handler - Collection offers bypass the master offer gate and minimum-offer floor

Description

The per-NFT offer path enforces the project's master `offers_enabled` flag and minimum-offer floor. The collection-offer path only checks granular collection-offer flags and `amount > 0`, so collection offers can remain active when offers are globally disabled and can be created for dust amounts.

Vulnerability Details

The `make_offer` handler constrains the project with both `project.enabled` and `project.offers_enabled`.

```
#[account(
        seeds = [Project::SEED, project.collection.as_ref()],
        bump = project.bump,
        constraint = project.enabled @ MarketplaceError::ProjectDisabled,
        constraint = project.offers_enabled @ MarketplaceError::OffersDisabled,
    )]
    pub project: Account<'info, Project>,
```

It also computes a minimum acceptable offer from `listing.price` and `project.min_offer_bps`.

```
let min_amount: u64 = (listing.price as u128)
        .checked_mul(project.min_offer_bps as u128)
        .ok_or(MarketplaceError::ArithmeticOverflow)?
        .checked_div(10_000)
        .ok_or(MarketplaceError::ArithmeticOverflow)?
        .try_into()
        .map_err(|_| error!(MarketplaceError::ArithmeticOverflow))?;
    require!(amount >= min_amount, MarketplaceError::OfferTooLow);
    require!(amount > 0, MarketplaceError::OfferTooLow);
```

The `make_collection_offer_v2` handler constrains only `project.enabled` at the account level.

```
#[account(
        seeds = [Project::SEED, project.collection.as_ref()],
        bump = project.bump,
        constraint = project.enabled @ MarketplaceError::ProjectDisabled,
    )]
    pub project: Account<'info, Project>,
```

The handler then checks only the granular offer surface and a nonzero amount.

```
if is_global {
        require!(project.global_offers_enabled,
MarketplaceError::GlobalOffersDisabled);
    } else {
        require!(project.specific_offers_enabled,
MarketplaceError::SpecificOffersDisabled);
    }
    // Validate amount
    require!(amount > 0, MarketplaceError::OfferTooLow);
```

There is no `project.offers_enabled` check and no equivalent of `min_offer_bps` for collection offers.

Impact

A project authority who disables offers through the master `offers_enabled` flag may still receive collection offers if the granular flags remain enabled. Buyers can also create 1-lamport collection offers, bypassing the marketplace's configured quality floor for offers. After project migration, this composes with zero-filled granular flags: manually re-enabling granular collection offers while leaving the master flag disabled still allows collection offers.

Recommendation

Require `project.offers_enabled` in `MakeCollectionOfferV2` in addition to the granular flags.

```
constraint = project.offers_enabled @ MarketplaceError::OffersDisabled
```

Add a collection-offer minimum, either as a marketplace-level lamport floor or a new project field such as `min_collection_offer_lamports`. If percentage-based collection floors are desired, they must be derived from a reliable collection floor oracle or omitted in favor of an explicit lamport minimum.

Status

Resolved

[M-08] src/instructions/buy_nft.rs#handler - pNFT purchases distribute SOL before buyer-funded token-record creation

Description

The `buy_nft` handler pays seller, fee wallets, and royalty recipients before transferring the NFT. For pNFTs, the later Metaplex transfer uses the buyer as payer for destination token-record creation, so buyers with exactly enough SOL for the listed price can fail after the fee transfers have reduced their remaining balance.

Vulnerability Details

The `buy_nft` handler calculates fees, then immediately transfers SOL from the buyer.

```
let fees = calculate_fees(
        price,
        effective_royalty_bps,
        marketplace.platform_fee_bps,
        project.project_fee_bps,
    )?;
    // 1. Transfer SOL: buyer -> seller (net proceeds)
    if fees.seller_receives > 0 {
        system_program::transfer(
            CpiContext::new(
                ctx.accounts.system_program.to_account_info(),
                system_program::Transfer {
                    from: ctx.accounts.buyer.to_account_info(),
                    to: ctx.accounts.seller.to_account_info(),
                },
            ),
            fees.seller_receives,
        )?;
    }
```

After seller, platform, project, and royalty transfers, it invokes the NFT transfer.

```
match nft_type {
        NFT_TYPE_PNFT => {
```

```
            buy_pnft(&ctx, authority_seeds)?;
        }
        NFT_TYPE_CORE => {
            buy_core(&ctx, authority_seeds)?;
        }
        NFT_TYPE_LEGACY => {
            buy_legacy(&ctx, authority_seeds)?;
        }
        NFT_TYPE_CNFT => {
            let params =
cnft_params.as_ref().ok_or(MarketplaceError::MissingCnftParams)?;
            buy_cnft(&ctx, authority_seeds, params)?;
        }
        _ => return Err(MarketplaceError::InvalidNftType.into()),
    }
```

The pNFT helper names the buyer as `payer`.

```
fn buy_pnft<'info>(
    ctx: &Context<'_, '_, '_, 'info, BuyNft<'info>>,
    authority_seeds: &[&[u8]],
) -> Result<()> {
    TransferV1CpiBuilder::new(&ctx.accounts.token_metadata_program.to_account_info())
        .token(&ctx.accounts.escrow_token_account.to_account_info())
        .token_owner(&ctx.accounts.marketplace_authority.to_account_info())
        .destination_token(&ctx.accounts.buyer_token_account.to_account_info())
        .destination_owner(&ctx.accounts.buyer.to_account_info())
        .mint(&ctx.accounts.mint.to_account_info())
        .metadata(&ctx.accounts.metadata.to_account_info())
        .edition(Some(&ctx.accounts.edition.to_account_info()))
        .token_record(Some(&ctx.accounts.escrow_token_record.to_account_info()))

.destination_token_record(Some(&ctx.accounts.buyer_token_record.to_account_info()))
        .authority(&ctx.accounts.marketplace_authority.to_account_info())
        .payer(&ctx.accounts.buyer.to_account_info())
        .system_program(&ctx.accounts.system_program.to_account_info())
        .sysvar_instructions(&ctx.accounts.sysvar_instructions.to_account_info())
```

```
        .spl_token_program(&ctx.accounts.token_program.to_account_info())
        .spl_ata_program(&ctx.accounts.associated_token_program.to_account_info())

.authorization_rules_program(Some(&ctx.accounts.authorization_rules_program.to_accoun
t_info()))

.authorization_rules(Some(&ctx.accounts.authorization_rules.to_account_info()))
        .amount(1)
        .invoke_signed(&[authority_seeds])?;
```

The sibling `accept_offer` handler documents and avoids this ordering issue by transferring the NFT first.

```
// ── NFT transfer FIRST (before fee distribution) ──
    // The pNFT TransferV1 CPI uses the seller as payer for token record
    // creation. Modifying seller/Offer-PDA lamports via fee distribution
    // before the CPI can trigger UnbalancedInstruction. Transfer the NFT
    // first so the CPI operates on unmodified account lamports. This also
    // keeps ordering consistent with accept_collection_offer_v2.
    let marketplace_key = marketplace.key();
    let authority_bump = ctx.bumps.marketplace_authority;
    let authority_seeds: &[&[u8]] = &[
        Marketplace::AUTHORITY_SEED,
        marketplace_key.as_ref(),
        &[authority_bump],
    ];
    match nft_type {
```

The `accept_collection_offer_v2` handler follows the same safer pattern.

```
// ── NFT transfer FIRST (before fee distribution) ──
    match nft_type {
        NFT_TYPE_PNFT => {
            transfer_pnft_seller_to_buyer(&ctx)?;
        }
```

Impact

Legitimate pNFT buyers whose wallet balance is exactly the listing price, or only slightly above it, can fail with an opaque Metaplex `UnbalancedInstruction` or payer-funding error. The transaction reverts atomically, so listing state is not corrupted and other buyers are not blocked. The impact is still meaningful because exact-balance checkout flows are common and the codebase already recognizes this failure mode in other handlers.

Recommendation

Move the NFT transfer block and `authority_seeds` initialization before any SOL distribution in `buy_nft`. Mirror the ordering and explanatory comment from `accept_offer`. Re-audit `buy_core_v2` and any future sale path for the same pattern.

Status

Resolved

Low

[L-01] src/instructions/update_project.rs#handler - min_offer_bps accepts values above 100 percent

Description

The `min_offer_bps` field is intended to represent a minimum offer as basis points of listing price, but project creation and update instructions accept any `u16`. Values above 10,000 silently make ordinary offers impossible.

Vulnerability Details

The `create_project` handler stores the caller-supplied value without a semantic upper bound.

```
pub fn handler(
    ctx: Context<CreateProject>,
    project_fee_bps: u16,
    enforce_royalties: bool,
    offers_enabled: bool,
    min_offer_bps: u16,
    default_offer_expiry: i64,
    cosigner: Option<Pubkey>,
) -> Result<()> {
    require_keys_eq!(
        ctx.accounts.authority.key(),
        ctx.accounts.marketplace.authority,
        MarketplaceError::Unauthorized
    );
    require!(
        project_fee_bps <= MAX_PROJECT_FEE_BPS,
        MarketplaceError::ProjectFeeTooHigh
    );
    require!(
        default_offer_expiry >= MIN_OFFER_EXPIRY_SECS
            && default_offer_expiry <= MAX_OFFER_EXPIRY_SECS,
        MarketplaceError::InvalidOfferExpiry
```

```
    );
    let project = &mut ctx.accounts.project;
    project.authority = ctx.accounts.authority.key();
    project.collection = ctx.accounts.collection.key();
    project.project_fee_bps = project_fee_bps;
    project.project_fee_wallet = ctx.accounts.authority.key();
    project.enforce_royalties = enforce_royalties;
    project.offers_enabled = offers_enabled;
    project.min_offer_bps = min_offer_bps;
```

The `update_project` handler has the same issue.

```
if let Some(min_bps) = min_offer_bps {
        project.min_offer_bps = min_bps;
    }
```

The offer path later interprets this field as `listing.price * min_offer_bps / 10_000`.

```
let min_amount: u64 = (listing.price as u128)
        .checked_mul(project.min_offer_bps as u128)
        .ok_or(MarketplaceError::ArithmeticOverflow)?
        .checked_div(10_000)
        .ok_or(MarketplaceError::ArithmeticOverflow)?
        .try_into()
        .map_err(|_| error!(MarketplaceError::ArithmeticOverflow))?;
    require!(amount >= min_amount, MarketplaceError::OfferTooLow);
```

If `min_offer_bps` is set to `60_000` instead of `6_000`, the minimum becomes 600 percent of the listing price. The program reports ordinary offers as `OfferTooLow`, not as a project misconfiguration.

Impact

A project authority can unintentionally disable per-NFT offers while believing they configured a normal offer floor. This is especially plausible because `u16` permits values up to 65,535 and basis-point fields elsewhere have explicit maximums. The resulting

failure mode is silent and user-facing: buyers see low-offer failures rather than an admin-facing configuration error.

Recommendation

Enforce `min_offer_bps <= 10_000` in both `create_project` and `update_project`.
Add `min_offer_bps <= 10_000` validation in both project setters, with a dedicated misconfiguration error.
Add a dedicated error so frontend and admin tooling can distinguish misconfiguration from an actually low offer.

Status

Resolved

[L-02] src/instructions/buy_nft.rs#BuyNft - sysvar_instructions is unchecked in several Metaplex CPI handlers

Description

Several sale and listing handlers pass `sysvar_instructions` to Metaplex CPIs as an unchecked account. The cNFT listing path pins the same account to the real instructions sysvar, showing the intended local pattern.

Vulnerability Details

The `buy_nft` account context declares the instructions sysvar as a bare unchecked account.

```
/// CHECK: Sysvar instructions
    pub sysvar_instructions: UncheckedAccount<'info>,
```

The same pattern appears in `list_nft`, `accept_offer`, `delist_nft`, `accept_collection_offer_v2`, and `accept_collection_offer_v2_listed`. By contrast, `list_cnft` uses an explicit address constraint.

```
/// CHECK: Sysvar instructions account — used to verify the companion
    /// Bubblegum transfer exists in the same transaction.
    #[account(address = anchor_lang::solana_program::sysvar::instructions::ID)]
    pub sysvar_instructions: UncheckedAccount<'info>,
```

Current pinned Metaplex dependencies validate the sysvar internally for the relevant CPI paths, so this is not currently an exploitable spoofing path. The issue is a local defense-in-depth inconsistency: the marketplace relies on downstream validation where it could cheaply enforce the invariant itself.

Impact

If future dependency versions relax validation, or if a new CPI path is added without downstream sysvar checks, these handlers could pass an attacker-controlled account into security-sensitive instruction-inspection logic. Under the currently pinned

dependencies, the likely impact is limited to clearer local validation and consistent error reporting.

Recommendation

Add the same address constraint used by `list_cnft` to all handlers that accept `sysvar_instructions`.

```
#[account(address = anchor_lang::solana_program::sysvar::instructions::ID)]
    pub sysvar_instructions: UncheckedAccount<'info>,
```

Status

Resolved

[L-03] src/instructions/accept_collection_offer_v2.rs#handler - Seller-controlled nft_type routes global collection-offer execution

Description

For global collection offers that accept any NFT type, the unlisted acceptance path trusts a seller-supplied `nft_type` argument to choose the transfer and royalty branch.

Vulnerability Details

The `accept_collection_offer_v2` handler takes `nft_type` as an instruction argument.

```
pub fn handler<'info>(
    ctx: Context<'_, '_, '_, 'info, AcceptCollectionOfferV2<'info>>,
    nft_type: u8,
    cnft_params: Option<CnftParams>,
    royalty_bps_override: Option<u16>,
) -> Result<()> {
```

The cNFT branch is blocked in this unlisted variant.

```
require!(
        nft_type != NFT_TYPE_CNFT,
        MarketplaceError::UnlistedCnftNotAllowed
    );
```

For typed offers, the handler checks the buyer's requested type.

```
if offer.nft_type != 255 {
        require!(
            nft_type == offer.nft_type,
            MarketplaceError::InvalidNftType
        );
    }
```

However, when `offer.nft_type == 255`, the seller controls the branch value. Later validation constrains the chosen branch through Token Metadata ownership, Core account ownership, Bubblegum restrictions, and Metaplex CPI semantics, so no

practical cross-standard royalty bypass was reproduced. The remaining issue is that the branch selector is still caller input rather than derived from the asset.

Impact

The current implementation is protected by downstream standard-specific checks, but the trust boundary is fragile. Future changes to CPI validation, metadata parsing, or supported standards could turn the caller-controlled branch into a real type confusion issue. It also makes the code harder to reason about because the program does not independently derive the NFT standard from on-chain state.

Recommendation

Derive `nft_type` from the supplied asset accounts instead of accepting it from the seller. For legacy and pNFT assets, inspect Token Metadata token standard. For Core assets, validate the MPL Core asset account. For cNFTs, route only through the listed cNFT path. Alternatively, disallow `255` any-type offers and require buyers to choose a concrete standard.

Status

Acknowledged

[L-04] src/instructions/update_project.rs#handler - Administrative state changes emit msg logs instead of structured events

Description

Several administrative and lifecycle handlers update marketplace-visible state but emit only `msg!()` logs. The program documents Anchor events as the authoritative feed for indexers.

Vulnerability Details

The program entry file describes events as the canonical indexing interface.

```
// Indexers should prefer parsing these via `EventParser` over scraping
// `msg!()` text logs. The `msg!()` calls stay in the handlers for
// human-readable debugging; the events are the authoritative feed.
```

However, administrative handlers such as `create_project`, `update_project`, `initialize`, `update_listing`, `transfer_project_authority`, `migrate_project`, `approve_cnft_list`, and `cancel_orphaned_offer` use only `msg!()` for state changes.

```
msg!("Project {} updated", project.collection);
```

```
if let Some(new_taker) = taker {
        listing.taker = new_taker;
        msg!(
            "Listing {} taker updated to {:?}",
            listing.mint,
            listing.taker
        );
    }
```

Structured events exist for sale and listing flows, but not for many admin changes that affect fees, project configuration, private-listing targets, cNFT approvals, and migration state.

Impact

Off-chain systems that follow the documented event feed can miss important state transitions. This is mostly observability and integration hygiene, but it becomes security-relevant when users or indexers need to detect fee changes, authority transfers, project disablement, or private-listing updates.

Recommendation

Add explicit events for administrative and lifecycle changes, including `MarketplaceInitialized`, `ProjectCreated`, `ProjectUpdated`, `ProjectAuthorityTransferred`, `ListingUpdated`, `ProjectMigrated`, `OrphanedOfferCancelled`, and `CnftListApproved`. Include old and new values for mutable fields where possible.

Status

Resolved

[L-05] src/state/offer.rs#Offer - Escrowed offers do not snapshot fee terms

Description

Offer PDAs store the escrowed amount and timing metadata but no platform or project fee snapshot. Accepted offers therefore settle under live fee values rather than the fee values visible when the buyer escrowed SOL.

Vulnerability Details

The `Offer` account contains no fee fields.

```
pub struct Offer {
    /// Buyer's wallet address
    pub buyer: Pubkey,
    /// Reference to the Listing PDA
    pub listing: Pubkey,
    /// NFT mint address (denormalized for queries)
    pub mint: Pubkey,
    /// Offer amount in lamports (escrowed in this PDA)
    pub amount: u64,
    /// Unix timestamp when offer expires
    pub expires_at: i64,
    /// Unix timestamp when offer was created
    pub created_at: i64,
    /// PDA bump
    pub bump: u8,
}
```

The `CollectionOffer` account has the same property.

```
pub struct CollectionOffer {
    /// Buyer's wallet address
    pub buyer: Pubkey,
    /// Reference to the Project PDA (collection)
    pub project: Pubkey,
    /// NFT mint address (Pubkey::default for global offers)
    pub mint: Pubkey,
    /// Offer amount in lamports (escrowed in this PDA)
```

```
    pub amount: u64,
    /// Unix timestamp when offer expires
    pub expires_at: i64,
    /// Unix timestamp when offer was created
    pub created_at: i64,
    /// NFT type required (0=Legacy, 1=pNFT, 2=Core, 3=cNFT, 255=any type for global)
    pub nft_type: u8,
    /// Whether this is a global collection offer (true) or specific NFT offer
(false)
    pub is_global: bool,
    /// PDA bump
    pub bump: u8,
    /// Reserved for future fields
    pub _reserved: [u8; 32],
}
```

Accept handlers calculate fees using the current marketplace and project accounts.

```
let fees = calculate_fees(
        price,
        effective_royalty_bps,
        marketplace.platform_fee_bps,
        project.project_fee_bps,
    )?;
```

The buyer's escrowed `amount` is fixed at offer creation, but the distribution of that amount among seller, platform, project, and creators is not fixed.

Impact

Routine fee changes by trusted operators can alter the economics of outstanding offers. Sellers accepting an old offer receive the net proceeds implied by current fees, not the net proceeds implied when the buyer created the offer. This can surprise sellers and make frontend offer displays stale.

Recommendation

Store `platform_fee_bps` and `project_fee_bps` in both `Offer` and `CollectionOffer` when the offer is created. Use these snapshots during acceptance. Apply the same pattern to `Listing` so both direct buys and offer accepts have stable fee semantics.

Status

Resolved

[L-06] src/instructions/buy_nft.rs#buy_core - v1 Core buy and delist helpers can be routed to delegate-mode Core listings

Description

The v1 Core helper paths assume escrow-mode Core custody, but Core assets listed through `list_core_v2` stay in the seller wallet with delegate plugins. Routing delegate-mode listings through the v1 buy or delist handlers produces avoidable failures and confusing client behavior.

Vulnerability Details

The v1 buy helper assumes the marketplace PDA is the current owner and transfers the Core asset from marketplace custody to the buyer.

```
fn buy_core<'info>(
    ctx: &Context<'_, '_, '_, 'info, BuyNft<'info>>,
    authority_seeds: &[&[u8]],
) -> Result<()> {
    CoreTransferV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info())
        .asset(&ctx.accounts.mint.to_account_info())
        .collection(Some(&ctx.accounts.core_collection.to_account_info()))
        .payer(&ctx.accounts.buyer.to_account_info())
        .authority(Some(&ctx.accounts.marketplace_authority.to_account_info()))
        .new_owner(&ctx.accounts.buyer.to_account_info())
        .system_program(Some(&ctx.accounts.system_program.to_account_info()))
        .invoke_signed(&[authority_seeds])?;
```

The v1 delist helper has the same escrow-mode assumption.

```
fn return_core<'info>(
    ctx: &Context<'_, '_, '_, 'info, DelistNft<'info>>,
    authority_seeds: &[&[u8]],
) -> Result<()> {
    CoreTransferV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info())
        .asset(&ctx.accounts.mint.to_account_info())
        .collection(Some(&ctx.accounts.core_collection.to_account_info()))
        .payer(&ctx.accounts.seller.to_account_info())
```

```
        .authority(Some(&ctx.accounts.marketplace_authority.to_account_info()))
        .new_owner(&ctx.accounts.seller.to_account_info())
        .system_program(Some(&ctx.accounts.system_program.to_account_info()))
        .invoke_signed(&[authority_seeds])?;
```

Delegate-mode Core listings created by `list_core_v2` do not transfer ownership to the marketplace PDA. They freeze the asset and grant plugin authority. Correct settlement and delist flows must thaw, transfer, and remove or revoke delegate plugins through `buy_core_v2` and `delist_core_v2`.

```
RevokePluginAuthorityV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info
())
            .asset(&ctx.accounts.mint.to_account_info())
            .collection(Some(&ctx.accounts.core_collection.to_account_info()))
            .payer(&ctx.accounts.seller.to_account_info())
            .authority(Some(&ctx.accounts.marketplace_authority.to_account_info()))
            .system_program(&ctx.accounts.system_program.to_account_info())
            .plugin_type(PluginType::FreezeDelegate)
            .invoke_signed(&[authority_seeds])?;
```

Impact

A client that sends a delegate-mode Core listing to the old `buy_nft` or `delist_nft` Core branch will likely fail inside the Core CPI because the asset is frozen and the marketplace PDA is not the asset owner. This is primarily a UX and routing issue rather than an asset-loss issue, because the correct v2 handlers remain available.

Recommendation

On the `NFT_TYPE_CORE` branches in `buy_nft` and `delist_nft`, reject delegate-mode listings explicitly.

Reject the v1 Core branches unless `listing.listing_mode() == LISTING_MODE_ESCROW`, so delegate-mode listings must use the v2 handlers.

This forces clients to use `buy_core_v2` and `delist_core_v2` for delegate-mode Core listings and gives a clear local error.

Status

Resolved

[L-07] src/utils.rs#read_core_royalties - Core royalties are read at sale time and can change before settlement

Description

Core NFT royalty configuration is read from MPL Core plugin state during sale execution. A Core update authority can modify royalty plugins in a prior transaction, changing the royalty BPS that a later buyer pays.

Vulnerability Details

The `read_core_royalties` function fetches the asset royalty plugin first and falls back to the collection plugin.

```
pub fn read_core_royalties(
    asset: &AccountInfo,
    collection: Option<&AccountInfo>,
) -> Result<Option<(u16, Vec<mpl_token_metadata::types::Creator>)>> {
    // Try the asset's own plugin first; if missing, fall back to the
    // collection's plugin. fetch_*_plugin returns Err with PluginNotFound
    // when the plugin isn't present — treat that as "no royalty config".
    let royalties: Option<Royalties> = match fetch_asset_plugin::<Royalties>(asset,
PluginType::Royalties) {
        Ok((_, r, _)) => Some(r),
        Err(_) => match collection {
            Some(coll) => fetch_collection_plugin::<Royalties>(coll,
PluginType::Royalties)
                .map(|(_, r, _)| r)
                .ok(),
            None => None,
        },
    };
    let Some(r) = royalties else { return Ok(None); };
```

Sale handlers call this helper at settlement time.

```
let (royalty_bps, creators) = if nft_type == NFT_TYPE_CORE {
        read_core_royalties(
```

```
            &ctx.accounts.mint.to_account_info(),
            Some(&ctx.accounts.core_collection.to_account_info()),
        )?.unwrap_or((0u16, vec![]))
```

Solana instruction execution is atomic, so there is no in-instruction interleaving after the sale starts. The issue is cross-transaction timing: if the Core update authority changes the royalty plugin in slot N and the buyer's purchase lands in slot N+1, the buyer pays the updated royalty configuration even if the listing was created under different terms.

Impact

Buyers and sellers cannot rely on Core royalty terms observed at listing time. A legitimate update authority can raise or lower royalties before settlement, changing buyer cost allocation and seller net proceeds. This is low severity because the update authority is part of the Core collection trust model, but the marketplace does not make this sale-time dependency explicit.

Recommendation

Snapshot Core royalty BPS and creators into the `Listing` account at list time. Use the snapshot at sale time for direct buys and listed offer accepts. If live Core royalty behavior is intentional, disclose it clearly in the frontend and event data.

Status

Resolved

[L-08] src/instructions/list_core_v2.rs#handler - All fetch_asset_plugin errors are treated as missing plugin state

Description

The `list_core_v2` handler treats every `fetch_asset_plugin` error as if the FreezeDelegate plugin is absent. This collapses real deserialization or account-state failures into the "plugin missing" branch.

Vulnerability Details

The handler attempts to read the Core asset's `FreezeDelegate` plugin.

```
let freeze_state = fetch_asset_plugin::<FreezeDelegate>(
        &ctx.accounts.mint.to_account_info(),
        PluginType::FreezeDelegate,
    );
    let (freeze_plugin_exists, pda_is_freeze_authority) = match &freeze_state {
        Ok((auth, _, _)) => {
            let pda_owns = matches!(
                auth,
                PluginAuthority::Address { address } if *address ==
marketplace_authority_key
            );
            (true, pda_owns)
        }
        Err(_) => (false, false),
    };
```

The comment says the intended absent case is `PluginNotFound`, but the code catches all errors. If the plugin exists but cannot be parsed or read due to account-state issues, the handler routes into the "add a fresh plugin" branch.

```
if freeze_plugin_exists {
        if pda_is_freeze_authority {
            // Our PDA already has authority (previous Orbis delist left the
            // plugin in place) — freeze directly.
```

```
UpdatePluginV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info())
                .asset(&ctx.accounts.mint.to_account_info())
                .collection(Some(&ctx.accounts.core_collection.to_account_info()))
                .payer(&ctx.accounts.seller.to_account_info())

.authority(Some(&ctx.accounts.marketplace_authority.to_account_info()))
                .system_program(&ctx.accounts.system_program.to_account_info())
                .plugin(Plugin::FreezeDelegate(FreezeDelegate { frozen: true }))
                .invoke_signed(&[authority_seeds])?;
            msg!("Asset frozen via existing FreezeDelegate (PDA already authority)");
        } else {
            // Plugin exists but a different authority owns it (ME/Tensor
            // leftover or owner-held) — transfer authority to our PDA, then
            // freeze.

ApprovePluginAuthorityV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_inf
o())
                .asset(&ctx.accounts.mint.to_account_info())
                .collection(Some(&ctx.accounts.core_collection.to_account_info()))
                .payer(&ctx.accounts.seller.to_account_info())
                .authority(Some(&ctx.accounts.seller.to_account_info()))
                .system_program(&ctx.accounts.system_program.to_account_info())
                .plugin_type(PluginType::FreezeDelegate)
                .new_authority(PluginAuthority::Address {
                    address: marketplace_authority_key,
                })
                .invoke()?;
            msg!("FreezeDelegate authority approved to marketplace PDA");

UpdatePluginV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info())
                .asset(&ctx.accounts.mint.to_account_info())
                .collection(Some(&ctx.accounts.core_collection.to_account_info()))
                .payer(&ctx.accounts.seller.to_account_info())

.authority(Some(&ctx.accounts.marketplace_authority.to_account_info()))
                .system_program(&ctx.accounts.system_program.to_account_info())
                .plugin(Plugin::FreezeDelegate(FreezeDelegate { frozen: true }))
                .invoke_signed(&[authority_seeds])?;
```

```
            msg!("Asset frozen via approved FreezeDelegate plugin");
        }
    } else {
        // No FreezeDelegate plugin — add a fresh one with marketplace PDA as
        // authority and frozen=true in one CPI.
        AddPluginV1CpiBuilder::new(&ctx.accounts.mpl_core_program.to_account_info())
            .asset(&ctx.accounts.mint.to_account_info())
            .collection(Some(&ctx.accounts.core_collection.to_account_info()))
```

That later CPI is likely to fail if the plugin actually exists, but the user receives an opaque downstream error rather than the true read failure.

Impact

The bug primarily affects diagnosability and defensive correctness. Transient or unexpected plugin-read failures are hidden until a later CPI fails, making Core listing errors harder to understand and increasing the chance that future refactors accidentally treat malformed plugin state as absent state.

Recommendation

Match only the specific "plugin not found" error as absence and return all other errors immediately.
Handle only the MPL Core plugin-not-found error as absent plugin state, and return every other `fetch_asset_plugin` error immediately.

Status

Resolved

[L-09] src/state/marketplace.rs#Marketplace - Marketplace has no global emergency pause

Description

The marketplace singleton has no global pause flag. Incident response must disable each project individually rather than halting all listing, buy, offer, and accept flows with one admin action.

Vulnerability Details

The `Marketplace` account stores the admin authority and platform fee configuration only.

```
pub struct Marketplace {
    /// Admin authority that can update platform fees
    pub authority: Pubkey,
    /// Platform fee in basis points (e.g., 200 = 2%)
    pub platform_fee_bps: u16,
    /// Wallet that receives platform fees
    pub platform_fee_wallet: Pubkey,
    /// PDA bump
    pub bump: u8,
}
```

There is no `paused: bool` field and no central account constraint in sale or offer handlers such as `constraint = !marketplace.paused`. The project model has `project.enabled`, but that is per collection.

Impact

During an incident that affects all projects, operators must submit one transaction per project to halt activity. This increases response time, operational complexity, and the chance that some projects remain active during mitigation. The impact is low because it is an emergency-control gap rather than a direct exploit path.

Recommendation

Add `paused: bool` to `Marketplace`, a `set_marketplace_pause(bool)` admin instruction, and `require!(!marketplace.paused, MarketplaceError::MarketplacePaused)` checks on all list, buy, offer-create, and offer-accept entry points. Emit a structured pause event.

Status

Resolved

[L-10] src/instructions/cancel_orphaned_offer.rs#handler - Orphaned-offer cancellation can close live offers without updating listing counters

Description

The `cancel_orphaned_offer` handler does not take or check a `Listing` account. A buyer can use it to cancel an offer on a still-live listing, bypassing the counter maintenance performed by `cancel_offer`.

Vulnerability Details

The account context contains the marketplace, the offer, and the buyer, but not the listing the offer references.

```
pub struct CancelOrphanedOffer<'info> {
    #[account(
        seeds = [Marketplace::SEED],
        bump = marketplace.bump,
    )]
    pub marketplace: Account<'info, Marketplace>,
    #[account(
        mut,
        close = buyer,
        seeds = [Offer::SEED, marketplace.key().as_ref(), offer.mint.as_ref(),
buyer.key().as_ref()],
        bump = offer.bump,
        has_one = buyer @ MarketplaceError::Unauthorized,
    )]
    pub offer: Account<'info, Offer>,
    #[account(mut)]
    pub buyer: Signer<'info>,
    pub system_program: Program<'info, System>,
}
```

The handler only logs and relies on Anchor's `close = buyer` to return rent and escrowed SOL.

```
pub fn handler(ctx: Context<CancelOrphanedOffer>) -> Result<()> {
```

```
    let offer = &ctx.accounts.offer;
    msg!(
        "Orphaned offer of {} lamports on NFT {} cancelled. Refunding buyer {}",
        offer.amount,
        offer.mint,
        offer.buyer
    );
    // Offer PDA is closed by Anchor (close = buyer)
    // Rent + escrowed SOL automatically returned to buyer
    Ok(())
}
```

The normal `cancel_offer` path updates listing counters.

```
let listing = &mut ctx.accounts.listing;
    listing.offer_count = listing.offer_count.saturating_sub(1);
    if cancelled_amount >= listing.highest_offer {
        listing.highest_offer = 0;
    }
```

Since `cancel_orphaned_offer` never proves the listing is gone, it can be used on a live offer and skip those updates.

Impact

Repeated make-offer and orphan-cancel cycles can inflate `listing.offer_count` and leave `listing.highest_offer` stale without keeping offers open. In the practical case, this can compose with the `u16` offer-count ceiling and eventually prevent new offers on a targeted listing.

Recommendation

Require proof that the listing PDA no longer exists. One approach is to add an optional unchecked listing account and reject if the expected listing PDA exists and is still owned by the program. Alternatively, split the instruction into a permissionless orphan cleanup path that derives and checks absence of the listing account.

Status

Resolved

[L-11] src/state/listing.rs#offer_count - Analytics counters can hit permanent overflow cliffs

Description

Several counters use checked addition and revert at their type maximum. `offer_count: u16` is the only practically reachable case and can be inflated through offer churn.

Vulnerability Details

The `Listing.offer_count` field is a `u16`.

```
pub struct Listing {
    /// Seller's wallet address
    pub seller: Pubkey,
    /// NFT mint address (for cNFTs: the asset ID from DAS)
    pub mint: Pubkey,
    /// Listing price in lamports
    pub price: u64,
    /// Reference to the Project PDA
    pub project: Pubkey,
    /// NFT type: 0=Legacy, 1=pNFT, 2=Core, 3=cNFT
    pub nft_type: u8,
    /// Number of active offers
    pub offer_count: u16,
    /// Highest active offer amount in lamports
    pub highest_offer: u64,
    /// Unix timestamp when listed
    pub created_at: i64,
    /// Optional private listing target. If Some, only this wallet can buy.
    pub taker: Option<Pubkey>,
    /// PDA bump
    pub bump: u8,
    /// Merkle tree address (cNFT only, zero for other types)
    pub merkle_tree: Pubkey,
    /// Leaf index in the Merkle tree (cNFT only, zero for other types)
    pub leaf_index: u32,
    /// Reserved for future fields (avoids realloc)
    pub _reserved: [u8; 28],
```

```
}
```

The `make_offer` handler increments it with `checked_add`.

```
let listing = &mut ctx.accounts.listing;
    listing.offer_count = listing
        .offer_count
        .checked_add(1)
        .ok_or(MarketplaceError::ArithmeticOverflow)?;
    if amount > listing.highest_offer {
        listing.highest_offer = amount;
    }
```

Project counters use checked addition as well.

```
project.active_listings = project
        .active_listings
        .checked_add(1)
        .ok_or(MarketplaceError::ArithmeticOverflow)?;
```

```
project.total_sales = project
        .total_sales
        .checked_add(1)
        .ok_or(MarketplaceError::ArithmeticOverflow)?;
```

The `active_listings: u32` and `total_sales: u32` counters are economically unlikely to reach their ceilings, but `offer_count: u16` can be reached with 65,535 offers. If paired with stale-counter paths, the count can become disconnected from live offers.

Impact

Once `offer_count` reaches `u16::MAX`, future `make_offer` calls revert with `ArithmeticOverflow`, preventing new offers for the listing. The counter appears to be display or analytics state rather than financial accounting, so reverting the whole offer path at the counter limit is unnecessary.

Recommendation

Widen `offer_count` to `u32` or use `saturating_add` for display-only counters. For `active_listings` and `total_sales`, prefer `saturating_add` unless there is a strict business reason to halt at the maximum.

Status

Resolved

[L-12] Cargo.toml#features - no-idl is defined but not enabled by default

Description

The crate defines Anchor's `no-idl` feature but does not include it in the default feature set. Deployed programs can therefore expose an unclaimed IDL authority surface unless operators explicitly handle IDL initialization.

Vulnerability Details

The `Cargo.toml` file declares the feature but leaves `default` empty.

```
[features]
no-entrypoint = []
no-idl = []
no-log-ix-name = []
cpi = ["no-entrypoint"]
default = []
idl-build = ["anchor-lang/idl-build", "anchor-spl/idl-build"]
```

If a program is deployed with an IDL account creation path available and no one claims or disables it, an external party may be able to publish a misleading IDL. This does not alter on-chain program logic, but it can affect clients and integrators that rely on IDL discovery.

Impact

Funds are not directly at risk, but SDK users, explorers, and integrators can be misled by an incorrect IDL if deployment operations do not claim the IDL authority promptly. This is a supply-chain and integration integrity issue.

Recommendation

Either include `no-idl` in the default feature set for production builds or create and control the IDL account immediately during deployment. Document the expected deployment process so the IDL authority state is intentional.

Status

Acknowledged

QA

[Q-01] src/state/project.rs#cosigner - Cosigner authority is scoped per project rather than per mint or tree

Description

The `project.cosigner` field is a single optional key trusted for cNFT listing approval and royalty overrides across the whole project. It is not bound on-chain to a specific mint, Merkle tree, or collection hashlist entry.

Vulnerability Details

The project stores a single cosigner.

```
/// Optional cosigner pubkey. When set, list/buy/accept_offer require this cosigner.
    pub cosigner: Option<Pubkey>,
```

The `approve_cnft_list` handler requires that one key to sign.

```
let required = project
        .cosigner
        .ok_or(MarketplaceError::CosignerRequired)?;
    require!(
        ctx.accounts.cosigner.key() == required,
        MarketplaceError::CosignerRequired
    );
```

The approval PDA records the mint, seller, project, and expiry, but the trust decision itself is delegated to the single cosigner key.

```
approval.mint = ctx.accounts.mint.key();
    approval.seller = ctx.accounts.seller.key();
    approval.project = ctx.accounts.project.key();
    approval.created_at = clock.unix_timestamp;
    approval.expires_at = clock
        .unix_timestamp
```

```
        .checked_add(APPROVAL_LIFETIME_SECS)
        .ok_or(MarketplaceError::ArithmeticOverflow)?;
```

The cNFT listing path then consumes the approval and verifies the companion Bubblegum transfer.

```
emit!(NftListed {
        mint: ctx.accounts.mint.key(),
        seller: ctx.accounts.seller.key(),
        project: ctx.accounts.project.key(),
        price,
        nft_type: NFT_TYPE_CNFT,
        listing_mode: LISTING_MODE_ESCROW,
        taker,
        created_at: clock.unix_timestamp,
    });
```

Impact

A compromised project cosigner can approve cNFT listings across the project scope. Removing the cosigner disables new cNFT approvals but does not necessarily give users a granular on-chain view of which mints, trees, or collections the cosigner was expected to cover. This is documented architecture, but integrators should understand the operational trust surface.

Recommendation

For stronger isolation, bind cosigner authority to a Merkle tree, collection root, or per-project allowlist stored on-chain. Document cosigner rotation and custody procedures, and emit events when the cosigner is added, removed, or changed.

Status

Acknowledged

[Q-02] src/instructions/initialize.rs#handler - First initializer becomes marketplace authority

Description

The initialization instruction assigns the marketplace authority to the first signer who initializes the singleton. There is no hardcoded deployer or expected authority check.

Vulnerability Details

The `Initialize` account context creates the singleton PDA at deterministic seeds.

```
#[account(
        init,
        payer = authority,
        space = Marketplace::SPACE,
        seeds = [Marketplace::SEED],
        bump,
    )]
    pub marketplace: Account<'info, Marketplace>,
```

The handler writes the caller's signer key as authority.

```
pub fn handler(ctx: Context<Initialize>, platform_fee_bps: u16) -> Result<()> {
    require!(
        platform_fee_bps <= MAX_PLATFORM_FEE_BPS,
        MarketplaceError::PlatformFeeTooHigh
    );
    let marketplace = &mut ctx.accounts.marketplace;
    marketplace.authority = ctx.accounts.authority.key();
    marketplace.platform_fee_bps = platform_fee_bps;
    marketplace.platform_fee_wallet = ctx.accounts.authority.key();
    marketplace.bump = ctx.bumps.marketplace;
```

If deployment operations do not initialize the marketplace immediately, the first successful initializer controls the marketplace.

Impact

This is an operational deployment risk rather than an ongoing runtime exploit once initialization is complete. A missed or delayed initialization step can allow the wrong signer to become marketplace authority.

Recommendation

Hardcode an expected deployer or authority pubkey for production builds, or ensure deployment tooling initializes the marketplace in the same operational sequence as program deployment. If a hardcoded key is not desired, document the initialization race explicitly.

Status

Acknowledged

[Q-03] src/instructions/migrate_project.rs#handler - Migration relies on raw byte offsets and unchecked project account input

Description

The `migrate_project` handler must read old project accounts that do not match the current Anchor layout, so it uses an unchecked account and manual byte offsets. The current checks make this safe in practice, but the code is fragile to future layout changes.

Vulnerability Details

The project account is unchecked at the Anchor account layer.

```
/// CHECK: Existing v1 project account. Owner, discriminator, and authority validated in handler.
    #[account(mut)]
    pub project: UncheckedAccount<'info>,
```

The handler compensates by checking program ownership and the Anchor discriminator.

```
require!(
        *project_info.owner == crate::ID,
        MarketplaceError::Unauthorized
    );
    let current_len = project_info.data_len();
    let target_len = Project::SPACE;
    // Already at v2 size — no-op
    if current_len >= target_len {
        msg!("Project already at v2 size ({}), skipping", current_len);
        return Ok(());
    }
    {
        let data = project_info.try_borrow_data()?;
        // Verify Project account discriminator. Using the Anchor-derived
        // constant so the check follows the struct definition automatically
        // rather than relying on a hand-transcribed byte array.
```

```
        require!(
            data.len() >= 40,
            MarketplaceError::Unauthorized
        );
        require!(
            data[..8] == Project::DISCRIMINATOR,
            MarketplaceError::Unauthorized
        );
```

It then reads the v1 authority from a hardcoded offset.

```
let stored_authority = Pubkey::try_from(&data[8..40])
            .map_err(|_| error!(MarketplaceError::Unauthorized))?;
        require!(
            stored_authority == ctx.accounts.authority.key(),
            MarketplaceError::Unauthorized
        );
```

The offset is correct for the known v1 layout, but it is a load-bearing assumption that is not encoded in a typed v1 struct.

Impact

Future edits to migration logic or assumptions about historical layouts could silently break authorization checks. The current implementation is acceptable for the known v1 layout, but the byte offsets and PDA derivation assumptions are not encoded as explicit invariants.

Recommendation

Define a small explicit v1 layout reader or constants for the historical discriminator, authority offset, collection offset, and target size. Re-derive the expected project PDA from the v1 collection bytes as an additional defense-in-depth check.

Status

Resolved

[Q-04] src/state/project.rs#default_offer_expiry - Default offer expiry is stored but never consumed

Description

The `default_offer_expiry` field is written during project creation and update, but offer-creation handlers require callers to pass explicit expiry values and never use the stored default.

Vulnerability Details

The project state includes a default expiry.

```
/// Default offer expiry in seconds (e.g., 172800 = 48h)
    pub default_offer_expiry: i64,
```

Project setters validate and write it.

```
project.default_offer_expiry = default_offer_expiry;
```

```
if let Some(expiry) = default_offer_expiry {
        require!(
            expiry >= MIN_OFFER_EXPIRY_SECS && expiry <= MAX_OFFER_EXPIRY_SECS,
            MarketplaceError::InvalidOfferExpiry
        );
        project.default_offer_expiry = expiry;
    }
```

Offer creation still requires `expiry_seconds` and validates that instruction argument directly.

```
pub fn handler(
    ctx: Context<MakeCollectionOfferV2>,
    amount: u64,
    expiry_seconds: i64,
    nft_type: u8,
    is_global: bool,
```

```
    nonce: u64,
) -> Result<()> {
```

Impact

The field expands the project configuration surface without affecting runtime behavior. Admins and frontend developers may assume that omitting or defaulting offer expiry uses the project setting, but the program does not implement that behavior.

Recommendation

Either remove `default_offer_expiry` or support optional expiry arguments that fall back to `project.default_offer_expiry`. Document whichever model is intended in the IDL and frontend.

Status

Acknowledged

Centralisation

The Orbis project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Orbis project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au